GATE
CS & DA

Artificial Intelligence

Un-informed Search

Lecture No. 3

By- SIDDHARTH SABHARWAL SIR

Recap of Previous Lecture

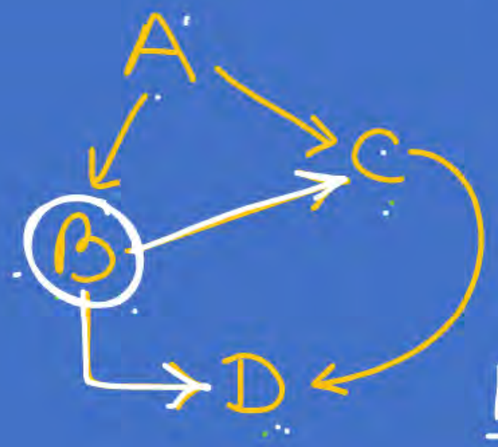
- Topic BFS
- Topic
- Topic
- Topic
- Topic

Topics to be Covered

- Topic DFS
- Topic
- Topic
- Topic
- Topic

Revision

BFS : ✓



Visited

A B

Queue

~~A~~ ~~B~~ ~~C~~ C D

FIFO

Skip

Depth first Search

- Depth-First Search (DFS) is a strategy for traversing or searching tree or graph structures.
- In the context of artificial intelligence (AI), DFS is often used for exploring possible states in search problems, finding solutions in game playing, planning, and more.

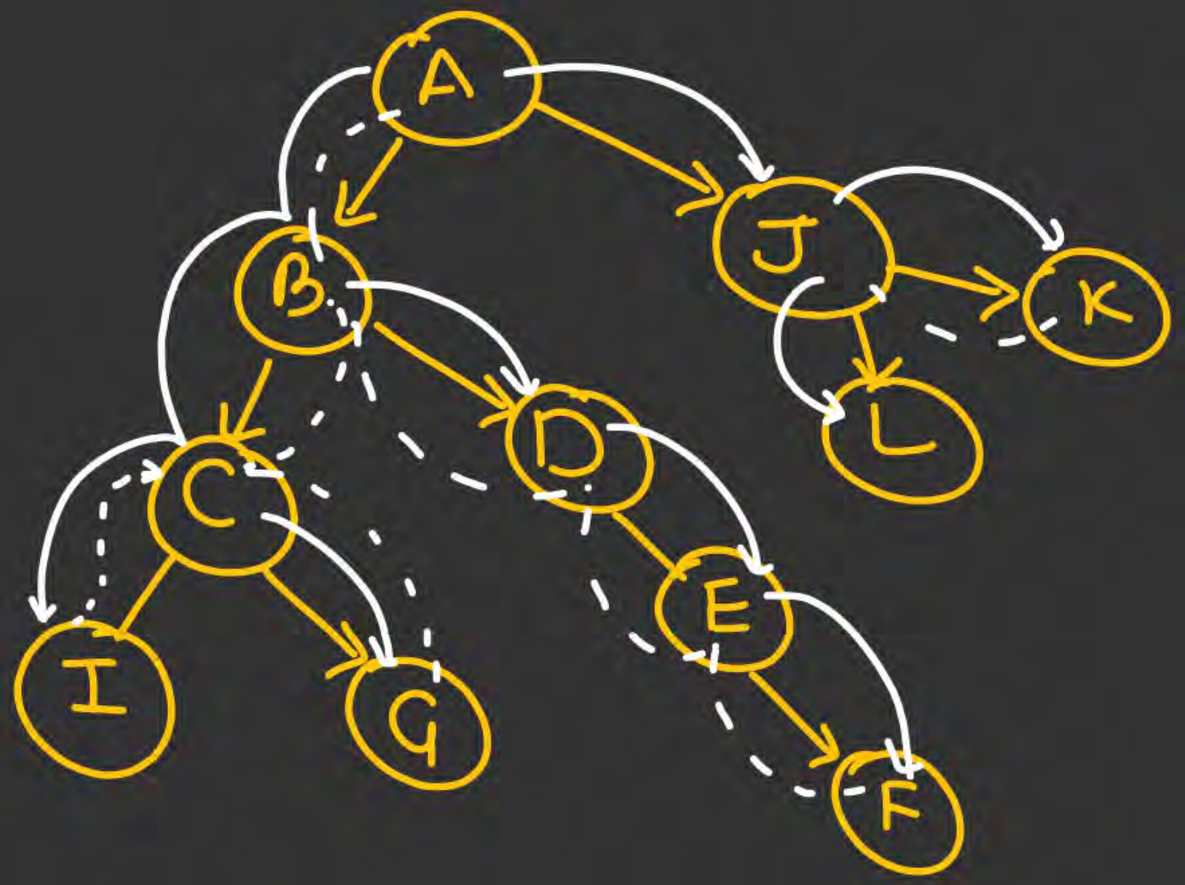
Step1 ⇒ two list Visited, Stack.

- Start with starting node and insert it into Stack.

2 → • now Remove the last node in stack, mark it as Visited and now insert neighbour node into stack.

3 → • Only check that no visited node is to be inserted,

- Repeate step 2,3 till goal node is Reached


A
B
J
K
L
C
D
E
I
G
F

Depth first Search

- Key Concepts
- Traversal Method: DFS explores as far as possible along each branch before backtracking.
- LIFO Structure: Uses a stack (Last-In-First-Out) data structure, either implicitly through recursion or explicitly.
- Completeness: DFS is not guaranteed to find a solution if one exists, especially in infinite search spaces.

```
        A
      / | \
    B   C   D
   /|  /|\  |\
  E F G H I J
 / \ |/|  \ |
K   L M   N O
```

Graph:

A -> [B, C, D]

B -> [A, E, F, C]

C -> [A, B, G, H, D]

D -> [A, C, I, J]

E -> [B, K, L]

F -> [B, G, M]

G -> [C, F, N]

H -> [C, I, O]

I -> [D, H]

J -> [D, O]

K -> [E]

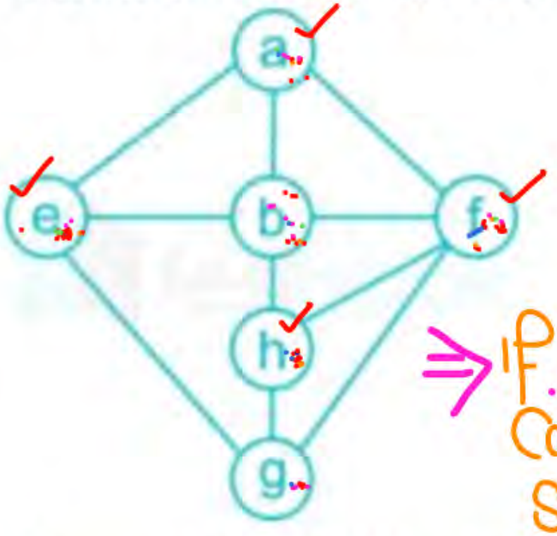
L -> [E]

M -> [F]

N -> [G]

O -> [H, J]

Consider the following graph.



Visited

A B E g f h

LIFO

Stack

~~A~~ f e ~~b~~ h f ~~A~~ ~~b~~ ~~h~~ ~~f~~ ~~A~~

⇒ If any node is cancelled from stack and entered into visited list then add all its neighbour except the visited node

Among the following sequences

I. a b e g h f

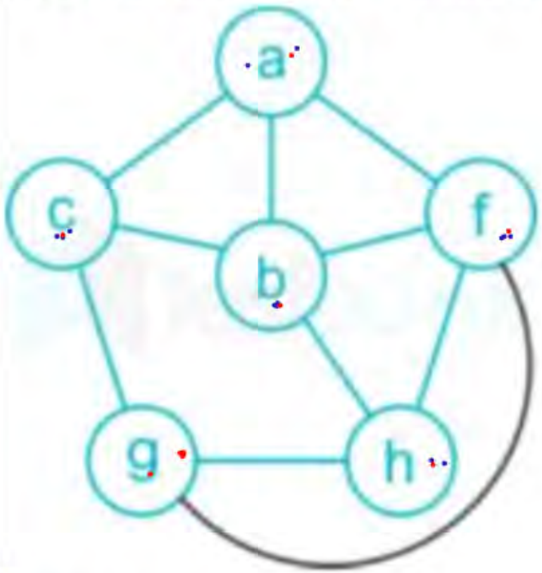
II. a b f e h g

III. a b f h g e

IV. a f g h b e

Which are depth first traversals of the above graph?

Consider the following graph:



⇒ alphabetical order

Visited: A B C g f H ←

Stack: ~~A~~ F ~~C~~ ~~B~~ H F ~~C~~ ~~g~~ H ~~f~~ ~~A~~

For the graph; the following sequences of depth first search (DFS) are given

(A) abcghf

(B) abfchg

(C) abfhgc

(D) afghbc

5 Path

Which of the following is correct?

Given below are two statements

Statement I : Breadth-First Search is optimal when all the step costs are equal whereas uniform-cost search is optimal with any step-cost.

Statement II : When all the step costs are same uniform-cost search expends more nodes at depth d than the Breadth-First Search.

In light of the above statements, choose the *correct* answer from the options given below

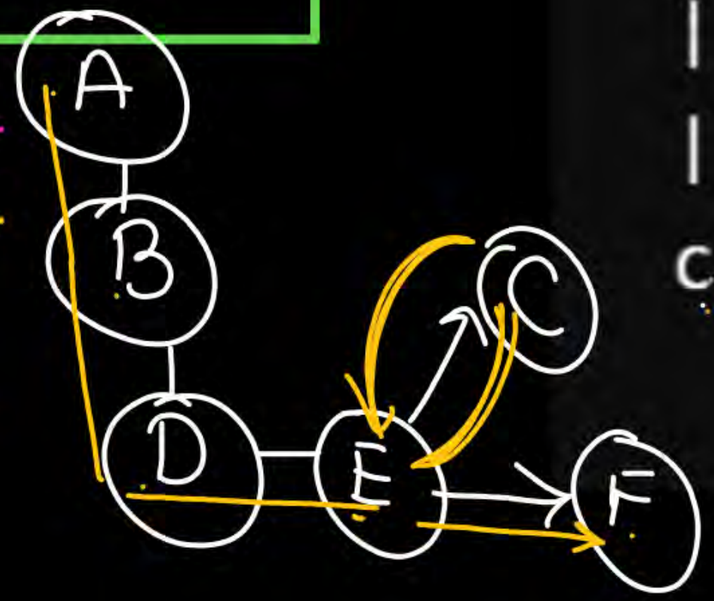
- From A to F.

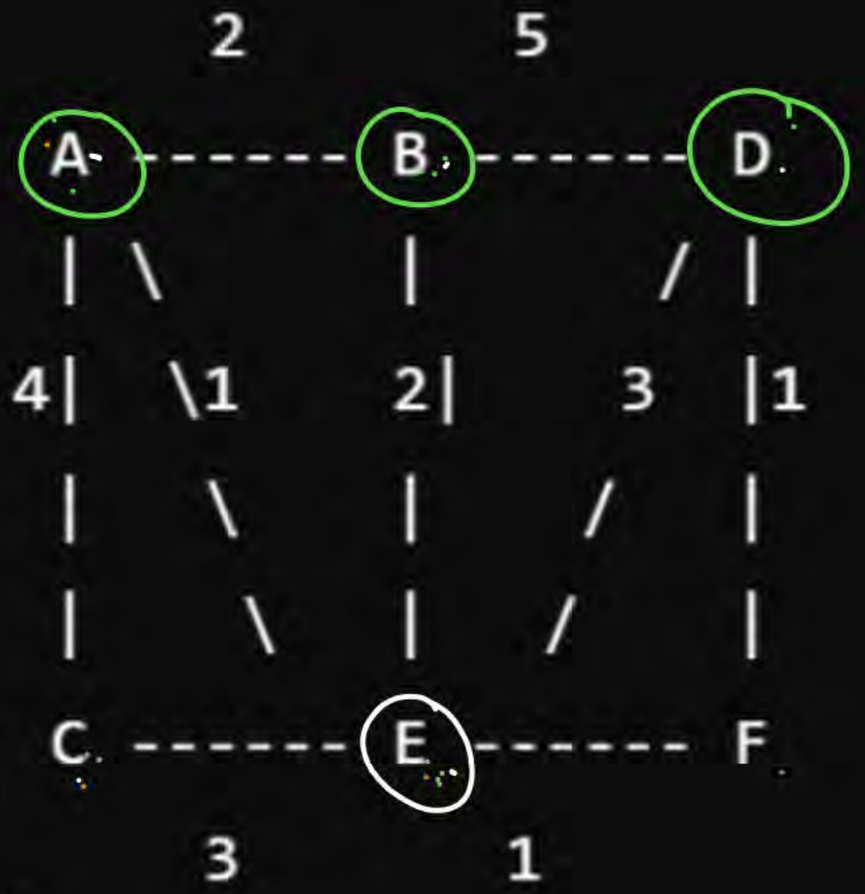
Visited: A B D E C F

(Back to back)

Stack: ~~A~~ E C ~~B~~ E ~~D~~ F ~~E~~ ~~F~~ ~~C~~ LIFO

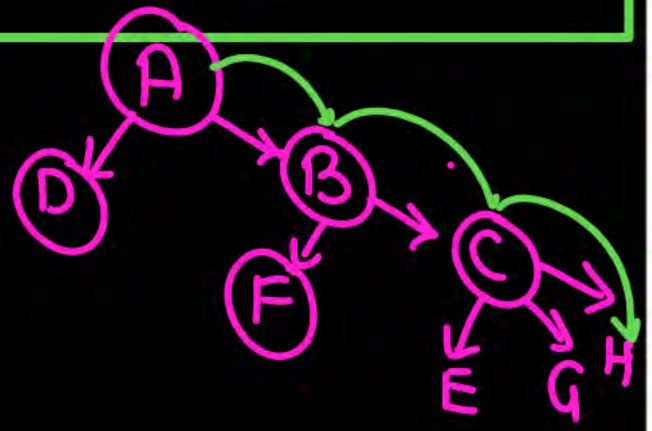
Path => ABDEF

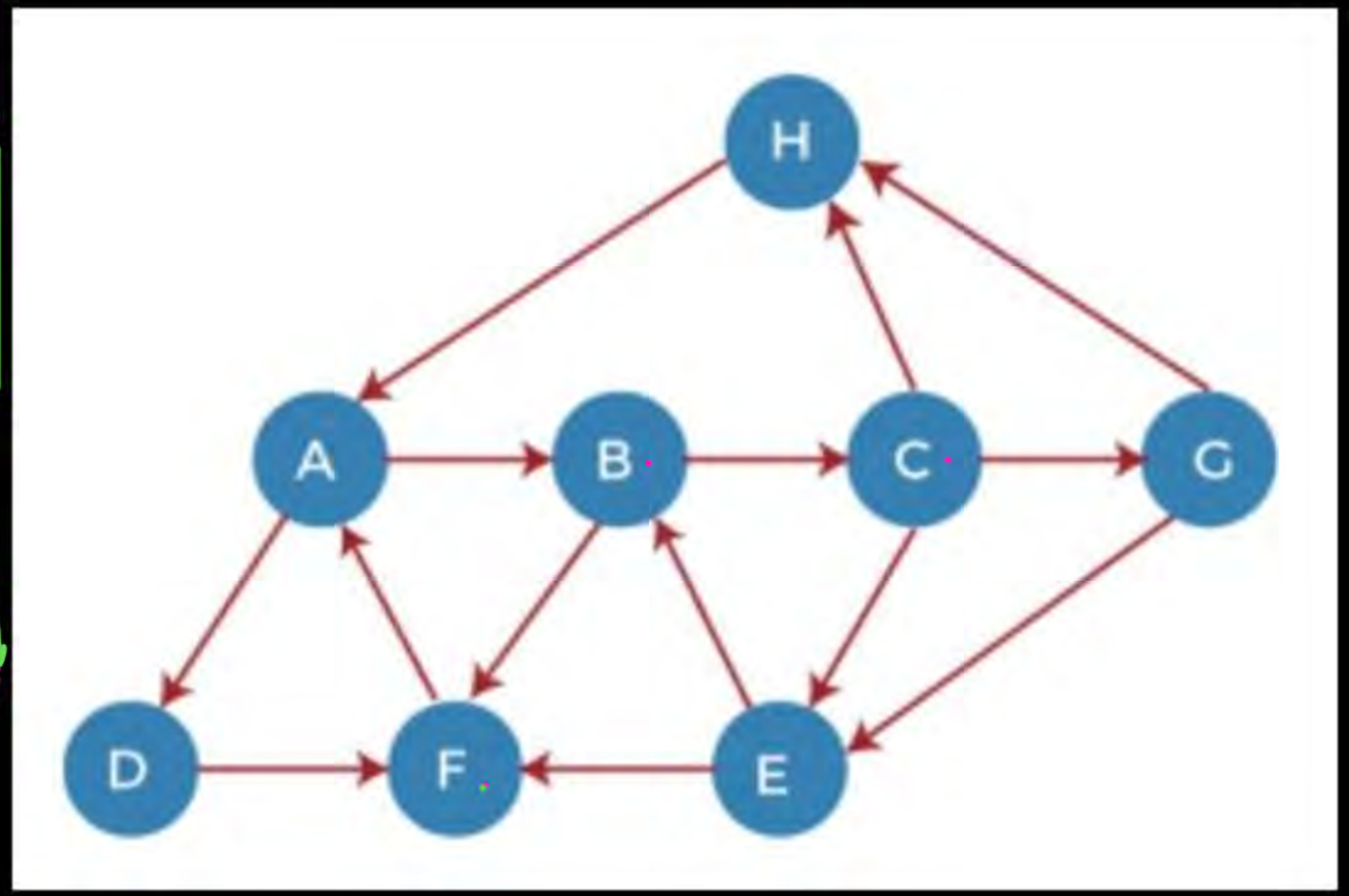




BFS

A B D C F E G H

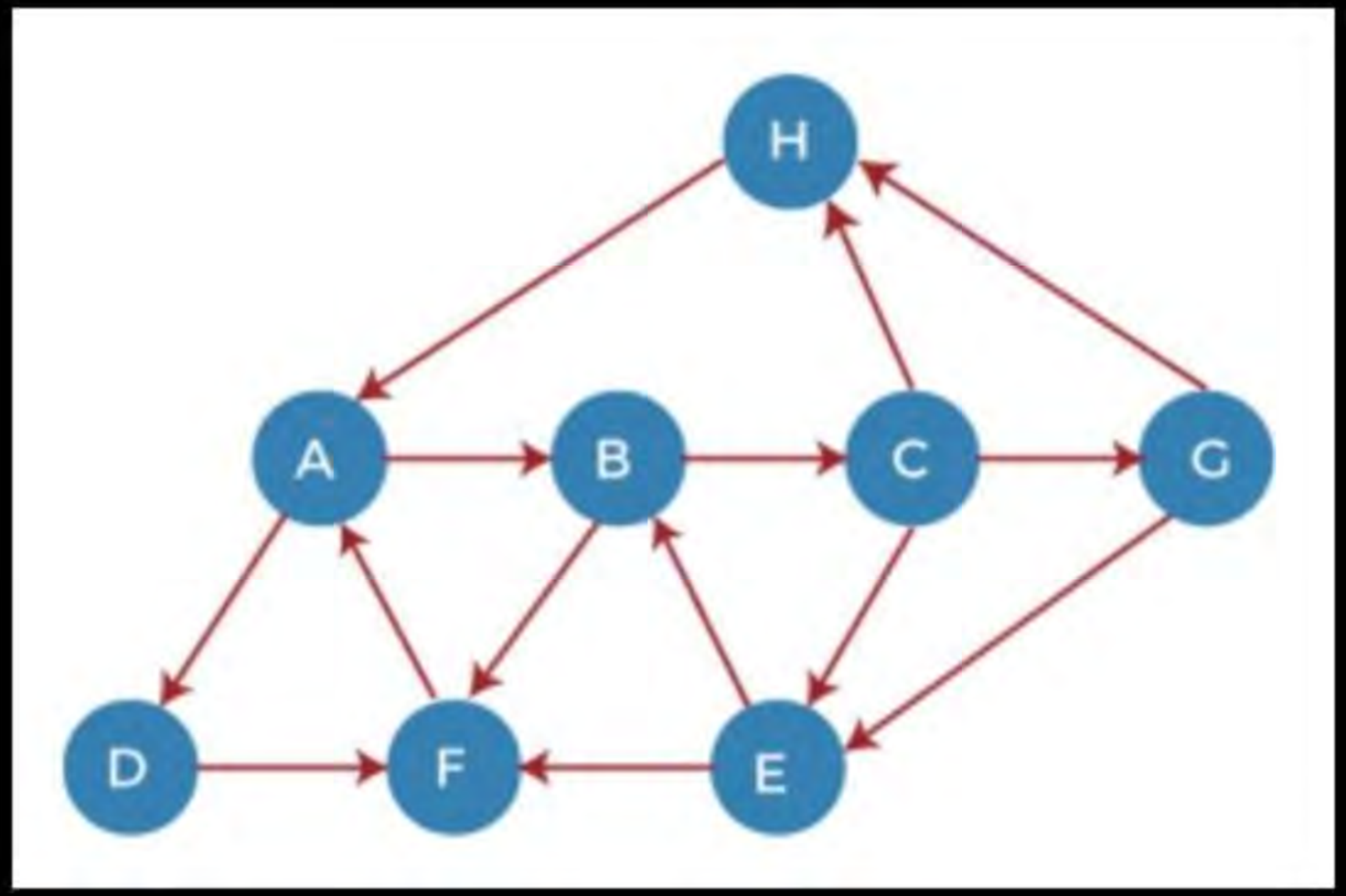




Q. DFS ⇒ tree ⇒

A → H

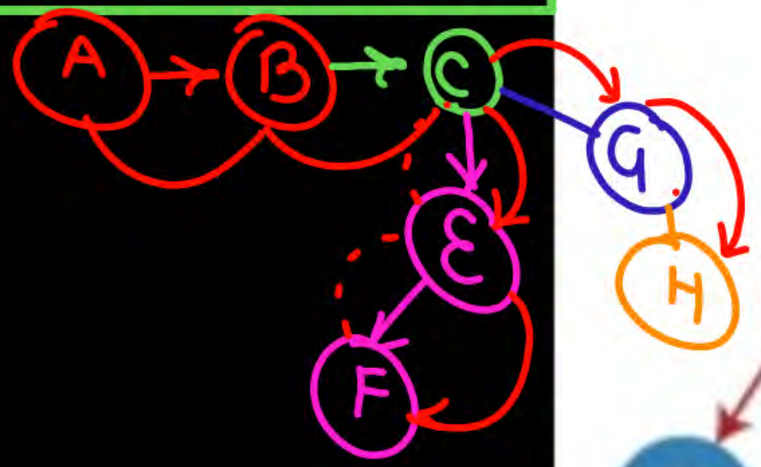
⇒ (alphabetical order)

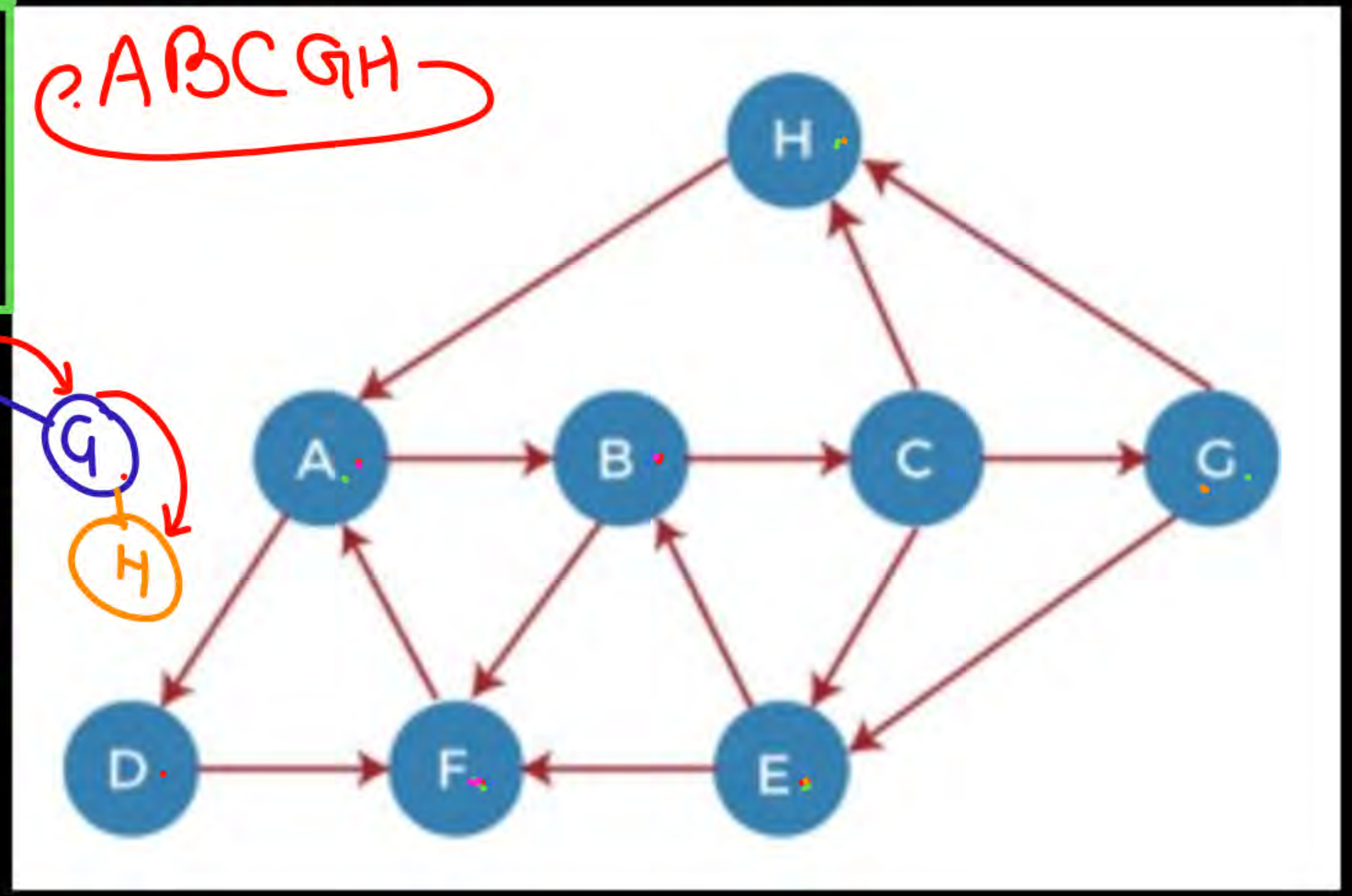


Visited
A B C E F G H.

Stack
~~A~~ D ~~B~~ F ~~C~~ H ~~G~~ ~~E~~ ~~A~~
~~A~~

ABCGH





Artificial Intelligence

M and K are the Goal state, solve using BFS and DFS.

BFS

⇒ BFS Stop @K

Visited

A B C D E F G H J K Stop

Queue

~~A~~ ~~B~~ ~~C~~ ~~D~~ E ~~F~~ ~~G~~ ~~H~~ ~~I~~ ~~J~~ K L M

Artificial Intelligence

M and K are the Goal state, solve using BFS and DFS.

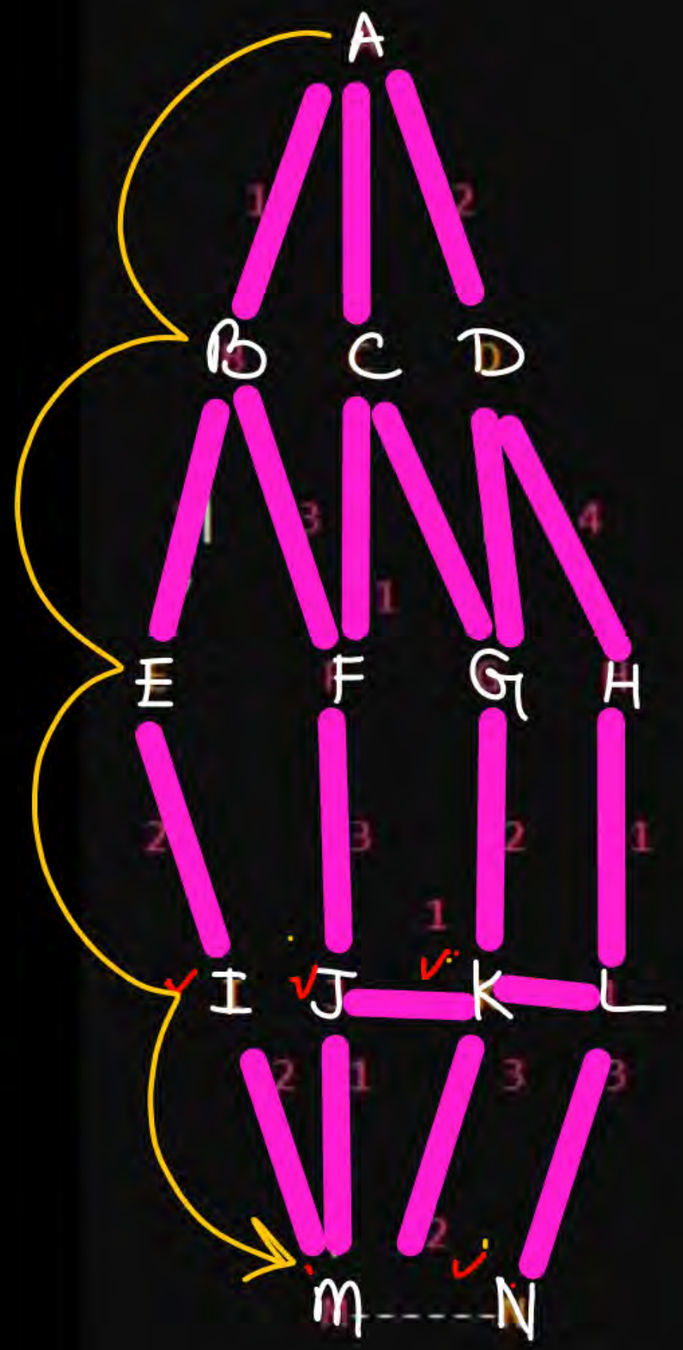
DFS

Visited

A B E I M | Stop

Stack

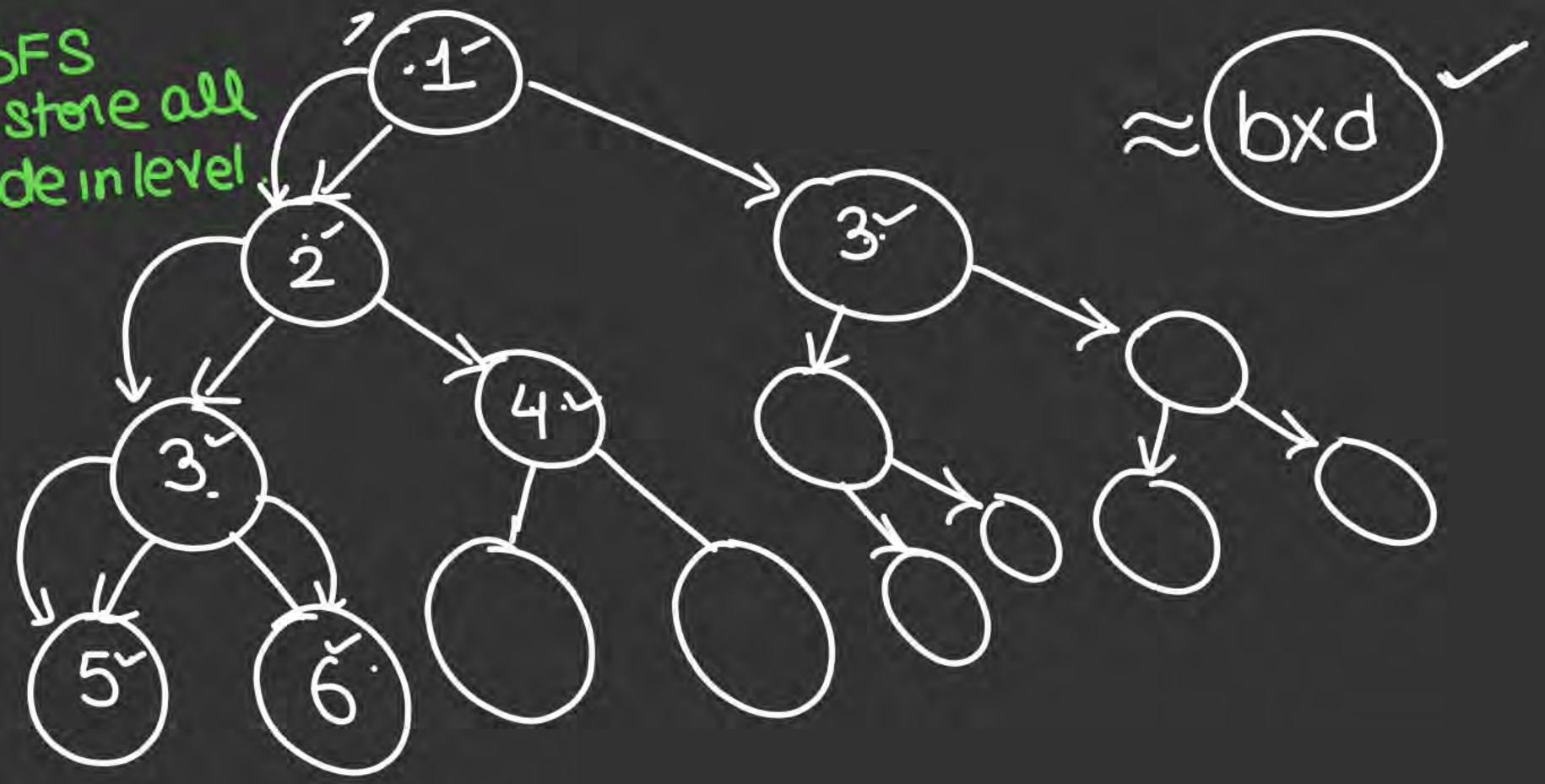
~~A~~ D C ~~B~~ F ~~E~~ ~~I~~ ~~M~~ N K J



Time Complexity ⇒ in worst Case scenario the algo has to check all nodes of graph

→ Same as BFS ⇒ $O(B^D)$

In BFS
we store all
node in level



Depth first Search

- Time and Space Complexity :

Completeness: DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$T(n) = 1 + n^2 + n^3 + \ldots\ldots\ldots + n^m = O(n^m)$

Where, m= maximum depth of any node and this can be much larger than d (Shallowest solution depth)

Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is **O(bm)**.

Depth first Search

Advantage:

- DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
- It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).

Disadvantage:

- There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
- DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

Depth Limited Search (DLS)

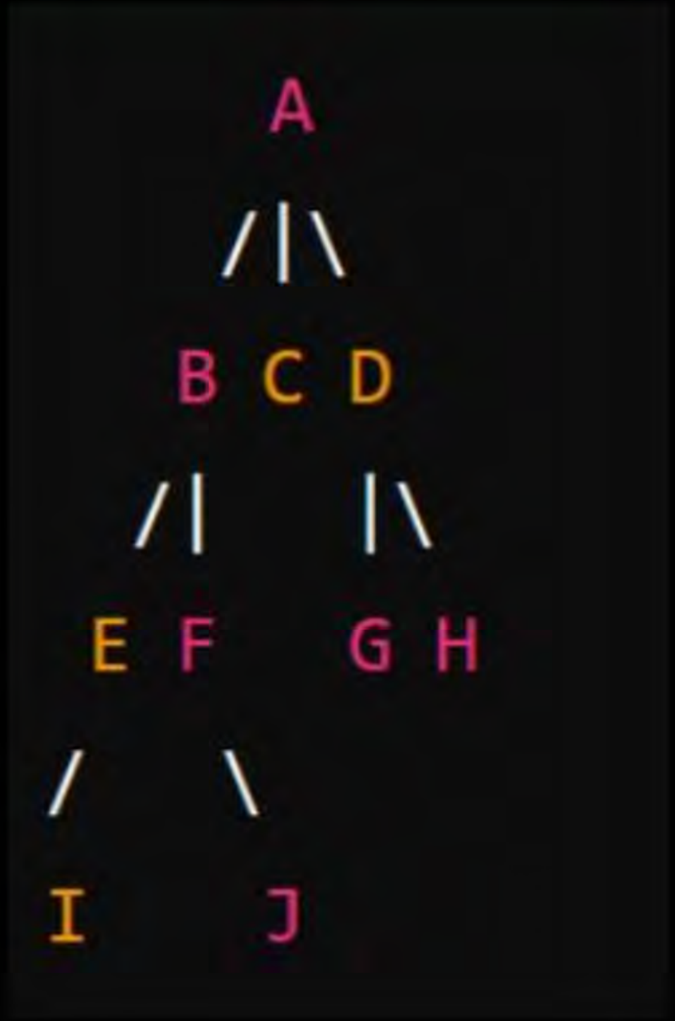
- Depth Limited Search is a modified version of DFS that imposes a limit on the depth of the search. This means that the algorithm will only explore nodes up to a certain depth, effectively preventing it from going down excessively deep paths that are unlikely to lead to the goal. By setting a maximum depth limit, DLS aims to improve efficiency and ensure more manageable search times.

Depth Limited Search (DLS)

- How Depth Limited Search Works
- Initialization: Begin at the root node with a specified depth limit.
- Exploration: Traverse the tree or graph, exploring each node's children.
- Depth Check: If the current depth exceeds the set limit, stop exploring that path and backtrack.
- Goal Check: If the goal node is found within the depth limit, the search is successful.
- Backtracking: If the search reaches the depth limit or a leaf node without finding the goal, backtrack and explore other branches.

Depth Limited Search (DLS)

Let's define our goal states as nodes G and J.
Apply DFS and DLS with depth limit 2.



Depth Limited Search (DLS)

- **Advantages of Depth Limited Search**
- Depth limited search is better than DFS and requires less time and memory space.
- DFS assures that the solution will be found if it exists infinite time.
- There are applications of DLS in graph theory particularly similar to the DFS.
- To combat the disadvantages of DFS, we add a limit to the depth, and our search strategy performs recursively down the search tree.
- **Disadvantages of Depth Limited Search**
- The depth limit is compulsory for this algorithm to execute.
- The goal node may not exist in the depth limit set earlier, which will push the user to iterate further adding execution time.
- The goal node will not be found if it does not exist in the desired limit.

Depth Limited Search (DLS)

- Performance Measures

- Completeness: The DLS is a complete algorithm in general except the case when the goal node is the shallowest node, and it is beyond the depth limit, i.e. $l < d$, and in this case, we never reach the goal node.
- Optimality: The DLS is a non-optimal algorithm since the depth that is chosen can be greater than d ($l>d$). Thus DLS is not optimal if $l > d$
- Time complexity is expressed as: It is similar to the DFS, i.e. $O(B^L)$, where L is the set depth limit.
- Space Complexity is expressed as: It is similar to DFS. $O(BL)$, where L is specified depth limit.

Iterative Deepening Search (IDS) or Iterative Deepening Depth First Search (IDDFS)

- There are two common ways to traverse a graph, BFS and DFS. Considering a Tree (or Graph) of huge height and width, both BFS and DFS are not very efficient due to following reasons.
- DFS first traverses nodes going through one adjacent of root, then next adjacent. The problem with this approach is, if there is a node close to root, but not in first few subtrees explored by DFS, then DFS reaches that node very late. Also, DFS may not find shortest path to a node (in terms of number of edges).
- BFS goes level by level, but requires more space.

Iterative Deepening Search (IDS) or Iterative Deepening Depth First Search (IDDFS)

- Key Concepts

- Depth-First Search (DFS): DFS explores as far as possible along each branch before backtracking. It uses a stack data structure (either implicitly through recursion or explicitly).
- Breadth-First Search (BFS): BFS explores all nodes at the present depth level before moving on to nodes at the next depth level. It uses a queue data structure.
- Iterative Deepening: IDDFS runs DFS repeatedly, increasing the depth limit each time until a solution is found. In the first iteration, it searches to depth 0, in the second to depth 1, and so on.

Iterative Deepening Search (IDS) or Iterative Deepening Depth First Search (IDDFS)

- Iterative Deepening Depth-First Search (IDDFS) combines the depth-first search's space efficiency with the breadth-first search's completeness. It repeatedly performs depth-limited searches with increasing depth limits until the goal is found.

```
        A
       / \
      B   C
     / \   \
    D   E   F
             \
              G
```

Iterative Deepening Search (IDS) or Iterative Deepening Depth First Search (IDDFS)

- The memory usage of Iterative Deepening Depth-First Search (IDDFS) is similar to that of Depth-First Search (DFS) because, at each depth limit, IDDFS essentially performs a DFS up to that depth. DFS's memory usage is primarily determined by the maximum depth of the stack.
- IDDFS has a memory usage of O(D), where dd is the depth of the goal node. This makes it space-efficient compared to Breadth-First Search (BFS), which uses $O(B^D)$ space.

Iterative Deepening Search (IDS) or Iterative Deepening Depth First Search (IDDFS)

Time Complexity:

Let's suppose b is the branching factor and depth is d then the worst-case time complexity is $O(b^d)$.

Space Complexity:

The space complexity of IDDFS will be **O(bd)**.

Iterative Deepening Search (IDS) or Iterative Deepening Depth First Search (IDDFS)

We will search for node N.

Iterative Deepening Search (IDS) or Iterative Deepening Depth First Search (IDDFS)

- Space Complexity
- Each iteration of depth-limited DFS uses space proportional to the depth limit.
- IDDFS does not store all nodes at a given depth level simultaneously; it only stores the path from the root to the current node.

Uniform Cost search

Uniform Cost Search (UCS) is a search algorithm used to find the least-cost path from a start node to a goal node in a graph. UCS is a variant of Dijkstra's algorithm and is used in situations where the path costs vary, making it more suitable for weighted graphs.

Uniform Cost search

Key Concepts

- Priority Queue: UCS uses a priority queue to explore the least-cost path first. Nodes are expanded in order of their cumulative cost from the start node.
- Cost: Each edge in the graph has a cost associated with it. UCS aims to find the path with the minimum cumulative cost.
- Expansion: The algorithm expands the node with the smallest cumulative cost, ensuring that the cheapest path is always chosen.

```
        A
      / | \
    1/  4| 3\
   B    C   D
  /|\  /|\  / \
2/ | 5|  | 1|  7\
E  F  G  H  I   J
 \  \ | /  /|  /
 3\  4|/  2| /5
   \  K   L  /
     \ | /  /
      M|/  /
      /|  /
     6|  3
     N   O
```

Time Complexity:

Let C* **is Cost of the optimal solution**, and **ε** is each step to get closer to the goal node. Then the number of steps is = C*/ε+1. Here we have taken +1, as we start from state 0 and end to C*/ε.

Hence, the worst-case time complexity of Uniform-cost search is $\mathbf{O(b^{1 + [C^*/\varepsilon]})}$/.

Space Complexity:

The same logic is for space complexity so, the worst-case space complexity of Uniform-cost search is $\mathbf{O(b^{1 + [C^*/\varepsilon]})}$.

Uniform-cost search is always optimal as it only selects a path with the lowest path cost.

Uniform Cost search

- Algorithm Steps
- Initialization: Initialize the priority queue with the start node, assigning it a path cost of 0. Set the cost of reaching the start node to 0.
- Expand Nodes:
- While the priority queue is not empty, remove the node with the lowest path cost from the queue.
- If the removed node is the goal node, return the path and the total cost.
- For the removed node, consider all its neighbors and calculate their path costs.
- If a neighbour has not been explored or a cheaper path to it is found, update its cost and add it to the priority queue.

Uniform Cost search

- Applications
- Pathfinding in artificial intelligence and robotics.
- Network routing protocols.
- Cost optimization problems in logistics and transportation.

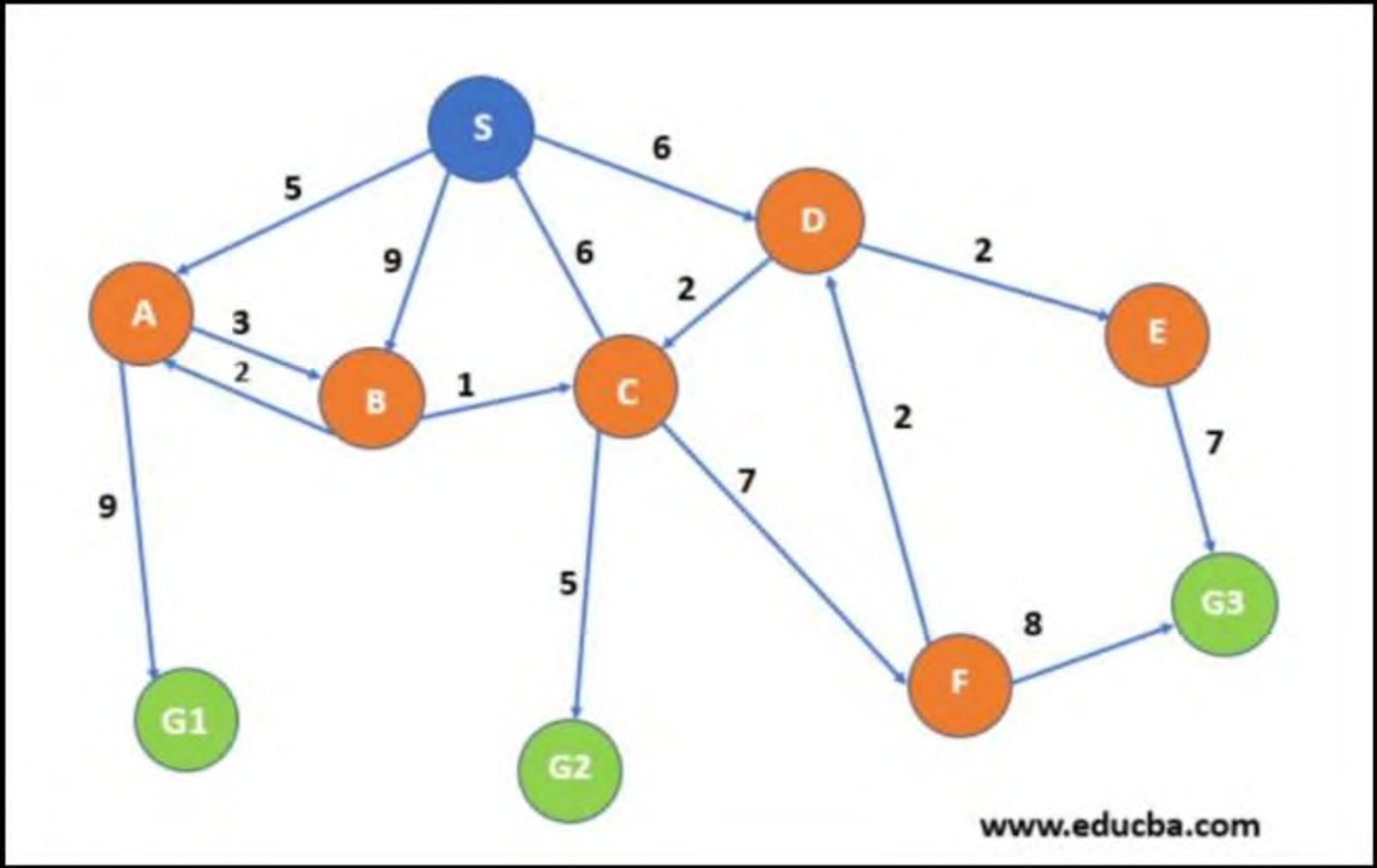
Uniform Cost search

Limitations

- UCS can be less efficient with large graphs due to its memory and time requirements.
- The algorithm assumes non-negative edge costs; negative costs can lead to infinite loops or incorrect results.

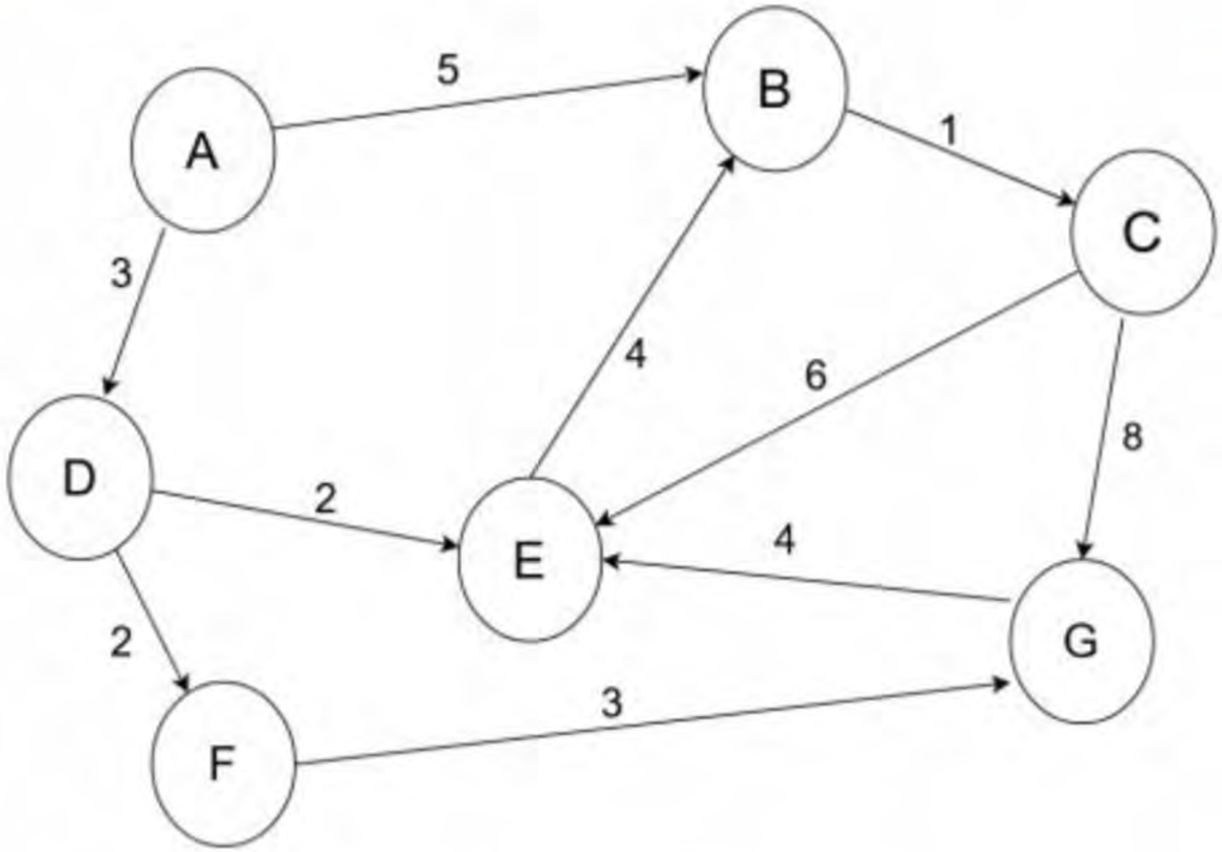
```
    4          2
(A)---(B)---(C)
 | \   |      |
6|  1\ |3    5|
 |    (D)---(E)
 |    / |     /|
 | 2/  |4   / |1
 | /   | /   /
(F)----(G)--(H)
    3      8
```

```
      4        3
A ------ B ------- C
| \     |        / |
|  \    |       /  |
|   2   1      4   2
|    \  |     /    |
D ----- E ---- F -- G
|  \    |    /     |
|   \   |   /      |
|    3  |  3       |
|     \ | /        |
H ----- I --------- J
      1      6
```


S
A
B
C
D
E
F
G1
G2
G3
5
6
9
6
2
2
3
2
1
2
7
7
9
5
8
www.educba.com

Uniform Cost search

Consider the following graph. Let the starting node be A and the goal node be G.



Finding path A to G using uniform cost search

```
          (4)
     S -------- A
    / \        / \
(2)/   \(3)  (2)   \(6)
  /     \   /       \
B        C           D
 \       / \         /
 (10)\(5)   \(3)  (4)
     \ /       \   /
      E         F
       \       /
       (2)\  (5)
          \ /
           G
```

THANK - YOU